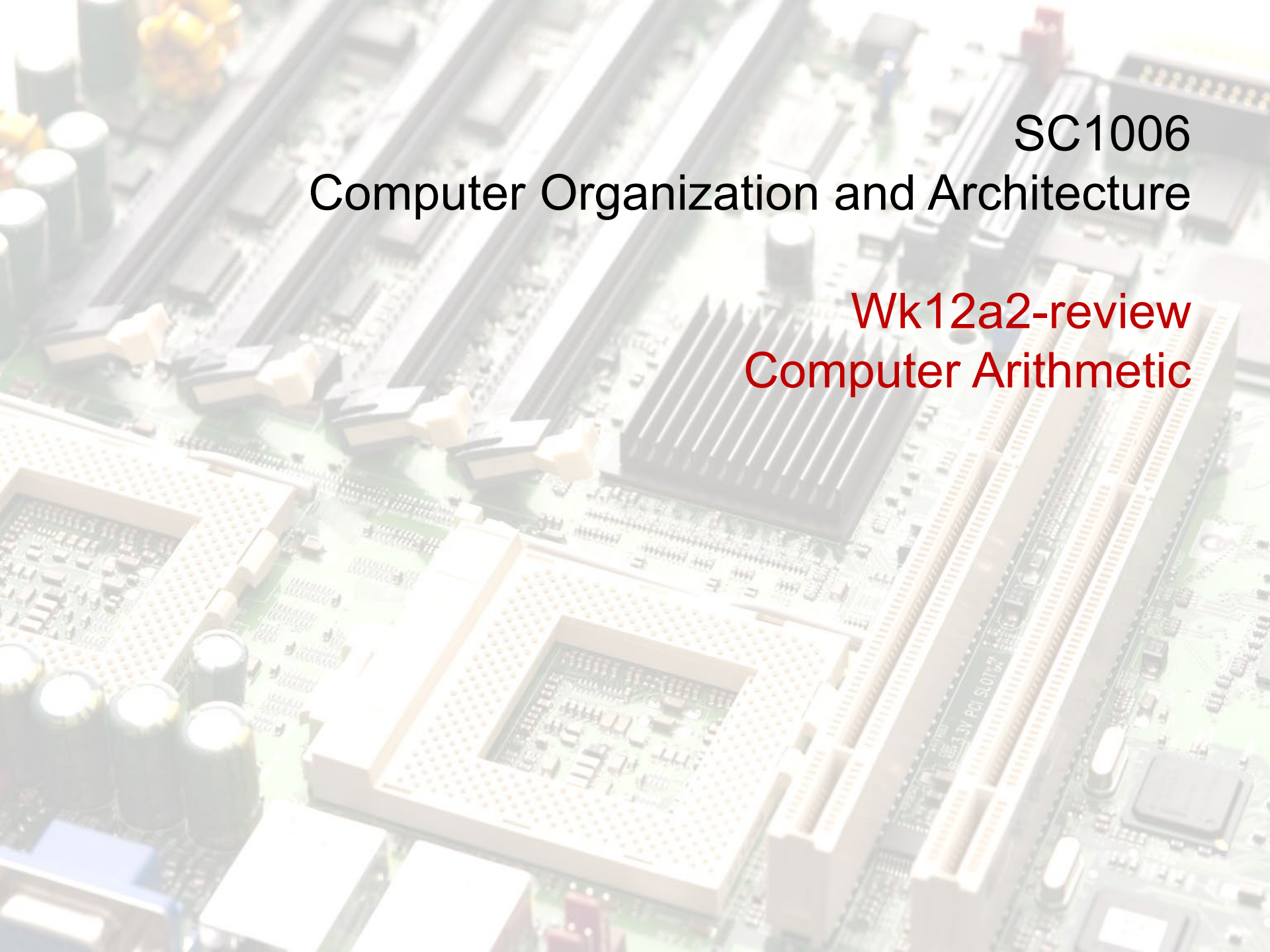


SC1006
Computer Organization and Architecture

Wk12a2-review
Computer Arithmetic

Major topics:

Positional Numbering

through increasing powers of a radix (or base)

Positive and Negative Numbers

Two's Complement Representation

same procedures for integer and fraction

e.g., 2's complement representation of $(-1.25)_{10}$ is 111110.11

Two's Complement To Decimal Conversion

Carry vs Overflow

Sign Extension

Multi-Precision Arithmetic

Fixed and Floating Point Numbers

IEEE754

Single Precision (32-bits)

Double Precision (64-bits)

We have learnt

Important issues & questions raised by some of you…

What is the 8-bit two's complement binary representation of -3?

A. -11
B. -011
C. 1101
✓ D. 1111 1101

3→0000 0011
Inv: 1111 1100
+1: 1111 1101

0x20-0x2 = ?

A. 0x18
B. 0x12
C. 0x1E
D. 0x30

0x20-0x2 = ?

Method 1:

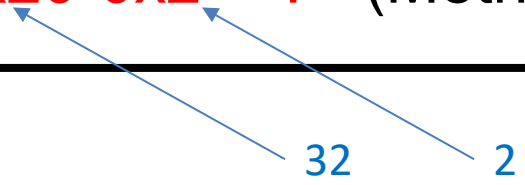
2’s complement of 0x2= Inv (0000 0010)+ 1 = 1111 1101 +1 = 1111 1110

0x20-0x2=0010 0000 + 1111 1110 =0001 1110 =0x1E

0x20-0x2 = ?

A. 0x18
B. 0x12
✓ C. 0x1E
D. 0x30

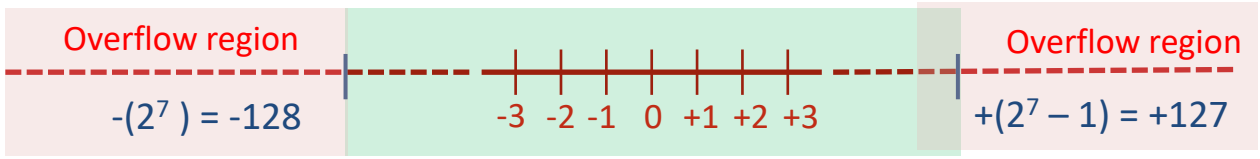
0x20-0x2 = ? (Method 2)



A. 0x18
B. 0x12
✓ C. 0x1E
D. 0x30

Carry vs Overflow Example

(illustrated with 8-bit representation with **2's complement**)



Example: 48 – 19 = 48 + (-19) = 29
First, negate 19 (00010011 -> 11101101)
Then add the two numbers and discard any carries emitting from the high order bit.

1		1	1					
	0	0	1	1	0	0	0	0
+	1	1	1	0	1	1	0	1
	0	0	0	1	1	1	0	1

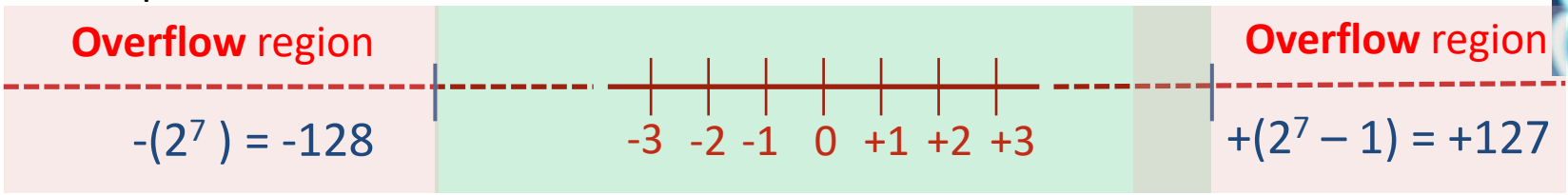
Carry vs. Overflow

- Unsigned Numbers
  - Carry = 1 always indicates an overflow (new value is too large to be stored in the given number of bits)
  - The overflow flag means nothing in the context of unsigned numbers
- Signed numbers
  - Overflow = 1 indicates an overflow
  - Carry flag can be set for signed numbers, but this does not necessarily mean an overflow has occurred.

4-bit representation (no problems if 5-bits

Expression	Result	Carry?	Overflow?	Correct Result?
0100 (+4) + 0010 (+2)	0110 (+6)	No	No	Yes
0100 (+4) + 0110 (+6)	1010 (-6)	No	Yes	No
1100 (-4) + 1110 (-2)	1010 (-6)	Yes	No	Yes
1100 (-4) + 1010 (-6)	0110 (+6)	Yes	Yes	No

8-bit representation:



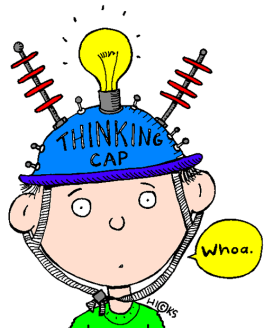
Detecting Overflow in Two's Complement Numbers

- Overflow can be easily detected by checking the Most Significant Bit (MSB) of the operands and result
- Conditions for overflow
  - In addition (Result = A + B)
    - If MSB(A) = MSB(B), and MSB(Result) ≠ MSB(A)
  - In subtraction (Result = A – B)
    - If MSB(A) ≠ MSB(B) and MSB(Result) ≠ MSB(A)

Operation	Conditions		Result
A + B	A > 0	B > 0	< 0
A + B	A < 0	B < 0	> 0
A – B	A > 0	B < 0	< 0
A – B	A < 0	B > 0	> 0

→ overflow

Re-think of Tut 1.5 (7)-(9)



Which of the following is/are true for overflow bit?

A. Overflow bit is set when a '1' is carried into the MSB of the result during computation

✓ B. To decide whether or not to set the overflow bit, the signed bits of the operands and result must be evaluated

✓ C. Overflow bit set implies that there is an error in the computation in a signed number system

✓ D. Overflow condition implies there is insufficient memory to contain the result

Overflow bit is only meaningful in a **signed** number system.

An overflow condition is a situation where the result of the computation falls into the overflow region of the number representation system. This can be detected by analysing the sign bits of the result and operands.

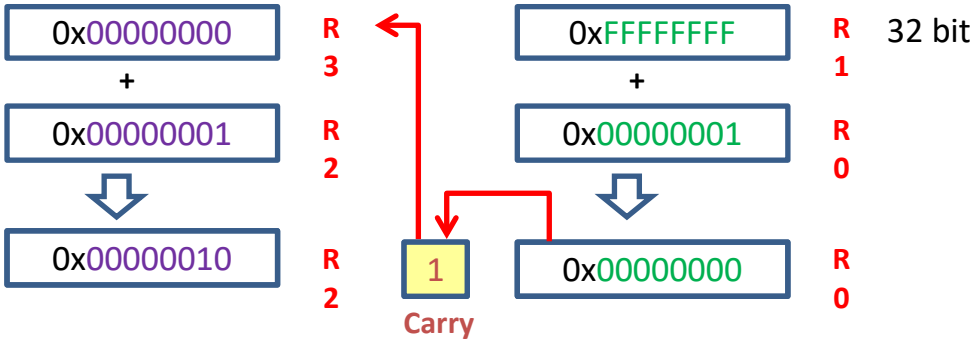
Effects of this condition are that the result is negative when adding two positive numbers, and result is positive when adding two negative numbers.

Multi-Precision Arithmetic

- How can we add operands that are larger than 32-bits (e.g. 64 bit operands) if we have only a single 32-bit ALU?
- The solution is to reuse the 32-bit adder for multi-precision addition
- Multi-precision arithmetic involves the computation of numbers whose precision is larger than what is supported by the maximum size of the processor register (Single-Precision)

```
  00000000 FFFFFFFF
+ 00000001 00000001
  --------------------
  00000002 00000000
  --------------------
```

```
ADD   R0,R0,R1  ; add lower word with carry out
ADC   R2,R2,R3  ; add upper word with carry in
```



Why do we only need to cater for 1 carry bit when performing a multi-precision addition? Why not 2 or 3 bits?

A. Only one carry bit is available in the processor

✓ B. Addition of 2 n-bit numbers will yield maximum (n+1) bit result.

C. Multiplication of 2 n-bit numbers will yield maximum (n+1) bit result.

D. Division of 2 n-bit numbers will yield maximum (n+1) bit result.

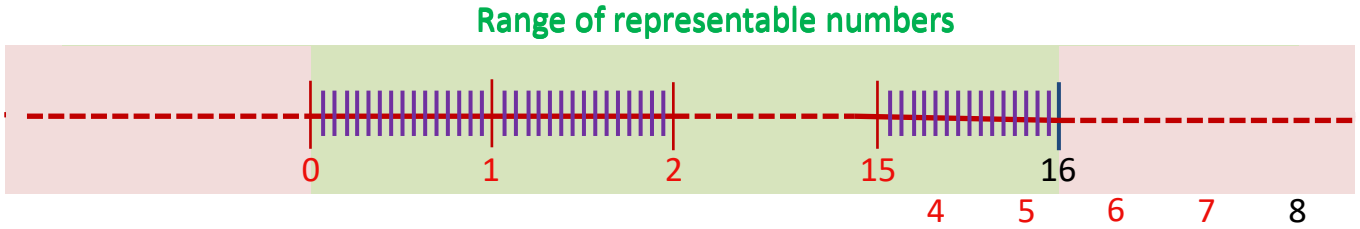
e.g. for an 8-bit unsigned number (carry is only meaningful for this) system, the range is 0 to 255. If you add two 8-bit number together, the latest result you will get is 255+255, i.e. the largest number multiply by 2.

In binary system, multiplying by two is equivalent to left shifting by 1 bit. This implies that the result will only have one more bit compared to the operands. Hence, only 1 bit will be carried out from the MSB of the result.

Range and Precision Trade-off

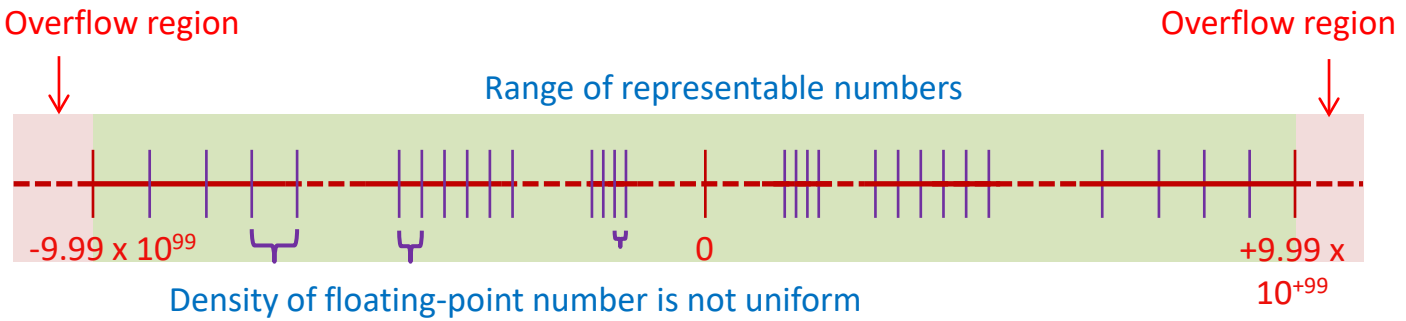
- What happen when the radix point moves/float from one end of a number to the other end?

2^3	2^2	2^1	2^0	•	2^{-1}	2^{-2}	2^{-3}	2^{-4}



- When radix point floats from LSB to MSB
  - Range of representable numbers reduces
  - Precision increases
- The example above illustrates the concept of fixed-point, but how do we represent a floating-point number?

Floating Point Representation



- Floating point representation can represent values across a wide range ($-9.99*10^{+99}$ to $+9.99*10^{+99}$).
- Size of exponent determines range of representable numbers.
- Size of mantissa and value of exponent determines the representable precision.
- Small numbers can be represented with good precision. When representing large numbers, precision can be sacrificed to achieve greater range.

Fixed Point vs Floating Point Number System

- Given the same number of bits to represent a data, e.g. 32 bits.
- Floating Point (IEEE754)
  - Max Range ≈ $2*2^{128}$ (-2^{128} to ~2^{128}).
  - Max Precision (near to zero) less than 2^{-126}
- Fixed Point
  - Max Range (Radix right of LSB, unsigned) ≈ 2^{32}
  - Max Precision (Radix left of MSB) 2^{-32}
- Floating point yield a larger range and better precision at small numbers with the same number of bits representation.
  - One usually needs the best precision when the numbers are small.
- However, Fixed point number has the advantage of having uniform precision across entire range.
  - Floating point number's precision changes across the range and the very coarse precision at the two end of the range may not be desirable to an intended algorithm.

What is the advantage of IEEE754 floating point representation over a fixed-point representation?

- ✓ A. Larger range than fixed point number given the same data bits representation
- B. Require less complex hardware to implement
- C. Allow uniform precision over the entire range
- ✓ D. Higher precision than a fixed-point number representation for small numbers, given the same data bits.

[A] E.g., IEEE754 single precision requires 32bit of data. This allows range of up to approximately +/- 2^128 as **exponential** value has 8 bits. Equivalent 32-bit fixed point number can only go up to max 2^32.
[B]Floating point computation is more complex than fixed point and typically requires special hardware such as FPU (floating-point unit) for more efficient computation.
[C] No, precision varies across the range.
[D] Precision of IEEE754 (normalised mode) near to the zero end of the range is 2^(-126), in comparison of this with a 32-bit fixed point number, if binary point is to the left of the MSB (the best precision for fixed point), it'll be 2^(-32).

Thank you!